`Looking up an entry`	O(log(n))	No
`Add a new entry`	O(n)	No
`Remove an entry`	O(n)	No

As you can see from table 4.2, in term of running time both the initial cost (to sort the list) and the cost for adding a new entry are pretty high. Also, linear extra memory might be needed if we need to make a copy of the original list, to preserve the original order.

4.4.3 Hash Table: Constant-Time on Average, Unless You Need Ordering

We introduced hash tables and hashing in appendix C; the main take away for hash tables is that they are used to implement associative arrays where the possible values to store come from a very large set (for instance, all possible strings or all integers), but normally we only need to store a limited number of them. If that's the case, then we use a hashing function to map the set of possible values (the *domain*, or source set) to a smaller set of `M` elements (the *codomain*, or target set), the indices of a plain array where we store the values associated to each key (as we explained in appendix C, we get to decide how big is `M` depending on some considerations on the expected performance). Typically, the set of values in the domain is referred to as